

Parallel & Distributed Computing

MPI Code Snippets

Contents

1	Code Snippets	2
1.1	Code Snippet #1	2
1.2	Code Snippet #2	3
1.3	Code Snippet #3	3
1.4	Code Snippet #4	4
1.5	Code Snippet #5	6
1.6	Code Snippet #6	7
1.7	Code Snippet #7	7
1.8	Code Snippet #8	8
1.9	Code Snippet #9	9
1.10	Code Snippet #10	10
1.11	Code Snippet #11	12
1.12	Code Snippet #12	13
1.13	Code Snippet #13	15
1.14	Code Snippet #14	16
1.15	Code Snippet #15	17
1.16	Code Snippet #16	18
1.17	Code Snippet #17	19
1.18	Code Snippet #18	20
1.19	Code Snippet #19	21
1.20	Code Snippet #20	22
2	Expected Outputs	23

1 Code Snippets

1.1 Code Snippet #1

Run with 3 processes

```
1 #include <mpi.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <unistd.h>
5
6 int main(int argc, char **argv)
7 {
8
9     int rank, size;
10    // Size = 3, in this case
11    /* Start up MPI */
12    MPI_Init(&argc, &argv);
13    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
14    MPI_Comm_size(MPI_COMM_WORLD, &size);
15
16
17    // MPI_SEND only copies value to system buf of the destination rank
18    // Data send by one MPI_Send can only be received by one MPI_RECV
19    // if two MPI_SEND calls are made to same rank, then 1 MPI_RECV will recv data
20    // from the first mpi_send, no matter how much you increase the count
21    // MPI_SEND and MPI_RECV work as a couple
22
23    int *sendBuf = (int *)malloc(2 * sizeof(int));
24    sendBuf[0] = 5;
25    sendBuf[1] = 7;
26    int recv[2] = {0, -1};
27
28    if (rank == 0){
29        MPI_Send(sendBuf, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);           // Send # 1
30        MPI_Send(sendBuf + 1, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);       // Send # 2
31        printf("Both of the data send\n");
32    }
33
34    else if (rank == 1){
35        sleep(5);
36        // The recv count must be >= send count, otherwise it'll throw error
37        MPI_Recv(recv, 2, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
38        // Only recv data from send#1
39        MPI_Recv(recv, 2, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
40        // Only recv data from send#2
41        printf("Data received in rank %d, data = %d, %d", rank, recv[0], recv[1]);
42    }
43    // MPI recv if the size is more than the data to be received then it ignores
44    // (doesn't pick up junk)
45    else if (rank == 2){
46        // this one will be blocked for eternity
47        MPI_Recv(&recv, 1, MPI_INT, MPI_ANY_SOURCE, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
48        printf("Data received in rank %d, data = %d, %d", rank, recv[0], recv[1]);
49    }
50
51
52    MPI_Finalize();
53
54    return 0;
55 }
```

1.2 Code Snippet #2

Run with 2 processes

```
1
2 #include <mpi.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <unistd.h>
6
7 int main(int argc, char **argv)
8 {
9
10     int rank, size;
11     // Size = 2, in this case
12     /* Start up MPI */
13     MPI_Init(&argc, &argv);
14     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
15     MPI_Comm_size(MPI_COMM_WORLD, &size);
16
17
18     int *sendBuf = (int *)malloc(2 * sizeof(int));
19     sendBuf[0] = 5;
20     sendBuf[1] = 7;
21     int recv[2] = {-1, -1};
22
23     if (rank == 0){
24         MPI_Send(sendBuf, 0, MPI_INT, 1, 0, MPI_COMM_WORLD);           // Send # 1
25         printf("Both of the data send\n");
26     }
27
28     else if (rank == 1){
29         sleep(5);
30         MPI_Status status;
31         int dataReceived;
32         // The recv count must be >= send count, otherwise it'll throw error
33         MPI_Recv(recv, 2, MPI_INT, 0, 0, MPI_COMM_WORLD, &status);
34         MPI_Get_count(&status, MPI_INT, &dataReceived);
35         printf("Count of data received = %d\n", dataReceived);
36         printf("Data received in rank %d, data = %d, %d", rank, recv[0], recv[1]);
37     }
38
39
40     MPI_Finalize();
41
42     return 0;
43 }
```

1.3 Code Snippet #3

Run with 3 processes

```
1 #include <mpi.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 int main(int argc, char **argv) {
6     MPI_Init(&argc, &argv);
7
8     int rank, size;
9     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
10    MPI_Comm_size(MPI_COMM_WORLD, &size);
11
12    int *send_data = (int *)malloc(size * sizeof(int));
13    int *recv_data = (int *)malloc(size * sizeof(int));
14
15    // Initializing Data
16    for (int i = 0; i < size; i++) {
17        send_data[i] = i + rank + i * rank;
18    }
```

```

19 printf("Process %d : \n", rank);
20 for (int i = 0; i < size; i++) {
21     printf("%d ", send_data[i]);
22 }
23 printf("\n");
24
25 // Array to store MPI_Request objects for non-blocking sends
26 MPI_Request *send_requests = (MPI_Request *)malloc(size * sizeof(MPI_Request));
27
28 // Send data to all processes (including itself)
29 for (int i = 0; i < size; i++) {
30     MPI_Isend(&send_data[i], 1, MPI_INT, i, 0, MPI_COMM_WORLD, &send_requests[i]);
31 }
32
33 // Receive data from all processes (including itself)
34 for (int i = 0; i < size; i++) {
35     MPI_Recv(&recv_data[i], 1, MPI_INT, i, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
36 }
37
38 // Wait for all non-blocking sends to complete
39 MPI_Waitall(size, send_requests, MPI_STATUSES_IGNORE);
40
41 // Print the received data
42 printf("Process %d received: ", rank);
43 for (int i = 0; i < size; i++) {
44     printf("%d ", recv_data[i]);
45 }
46 printf("\n");
47
48 MPI_Finalize();
49 return 0;
50 }

```

1.4 Code Snippet #4

Run with 3 processes

```

1
2 #include <mpi.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <unistd.h>
6
7 #define NUM_TASKS 5
8 #define TASK_DATA_SIZE 3
9 #define TERMINATION_TAG 2
10
11 int main(int argc, char **argv) {
12     MPI_Init(&argc, &argv);
13
14     int rank, size;
15     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
16     MPI_Comm_size(MPI_COMM_WORLD, &size);
17
18     if (size < 2) {
19         fprintf(stderr, "This program requires at least 2 processes.\n");
20         MPI_Abort(MPI_COMM_WORLD, 1);
21     }
22
23     if (rank == 0) {
24         // Master process
25         int task_data[NUM_TASKS][TASK_DATA_SIZE] = {
26             {1, 2, 3},
27             {4, 5, 6},
28             {7, 8, 9},
29             {10, 11, 12},
30             {13, 14, 15}
31         };
32
33         MPI_Request send_requests[NUM_TASKS];

```

```

34 MPI_Request recv_requests[NUM_TASKS];
35 int recv_buffer[NUM_TASKS][TASK_DATA_SIZE];
36 int completed_tasks = 0;
37
38 // Send tasks to workers
39 for (int i = 0; i < NUM_TASKS; i++) {
40     int worker_rank = (i % (size - 1)) + 1; // Distribute tasks among workers
41     MPI_Isend(task_data[i], TASK_DATA_SIZE, MPI_INT, worker_rank, 0, MPI_COMM_WORLD,
42             &send_requests[i]);
43     printf("Master sent task %d to worker %d\n", i, worker_rank);
44 }
45
46 // Post nonblocking receives to get results from workers
47 for (int i = 0; i < NUM_TASKS; i++) {
48     MPI_Irecv(recv_buffer[i], TASK_DATA_SIZE, MPI_INT, MPI_ANY_SOURCE, 1,
49             MPI_COMM_WORLD, &recv_requests[i]);
50 }
51
52 // Wait for all tasks to complete
53 while (completed_tasks < NUM_TASKS) {
54     int index;
55     MPI_Status status;
56     // wait if any of the requests is completed
57     MPI_Waitany(NUM_TASKS, recv_requests, &index, &status);
58
59     int count;
60     MPI_Get_count(&status, MPI_INT, &count);
61
62     printf("Master received result from worker %d: ", status.MPI_SOURCE);
63     for (int j = 0; j < count; j++) {
64         printf("%d ", recv_buffer[index][j]);
65     }
66     printf("\n");
67     completed_tasks++;
68 }
69
70 // Wait for all send operations to complete
71 MPI_Waitall(NUM_TASKS, send_requests, MPI_STATUSES_IGNORE);
72
73 // Send a termination message to each worker
74 for (int worker = 1; worker < size; worker++) {
75     // Send an empty message with TERMINATION_TAG to indicate no more tasks.
76     // See the working of send count = 0, how beneficial it is
77     MPI_Send(NULL, 0, MPI_INT, worker, TERMINATION_TAG, MPI_COMM_WORLD);
78     printf("Master sent termination message to worker %d\n", worker);
79 }
80 } else {
81     // Worker processes
82     MPI_Status status;
83     int task_data[TASK_DATA_SIZE];
84     int result_data[TASK_DATA_SIZE];
85
86     while (1) {
87         // Probe for incoming tasks or termination signal
88         // Blocking call, only use if you know a send is called
89         // Otherwise can use Irecv
90         MPI_Probe(0, MPI_ANY_TAG, MPI_COMM_WORLD, &status);
91
92         // Check if the incoming message is a termination message
93         if (status.MPI_TAG == TERMINATION_TAG) {
94             // Receive the termination message and break out of the loop.
95             MPI_Recv(NULL, 0, MPI_INT, 0, TERMINATION_TAG, MPI_COMM_WORLD,
96                     MPI_STATUS_IGNORE);
97             printf("Worker %d received termination signal. Exiting.\n", rank);
98             break;
99         }
100
101         int count;
102         MPI_Get_count(&status, MPI_INT, &count);
103         MPI_Recv(task_data, count, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);

```

```

101     printf("Worker %d received task: ", rank);
102     for (int i = 0; i < count; i++) {
103         printf("%d ", task_data[i]);
104     }
105     printf("\n");
106
107     // Process the task (e.g., multiply each element by 2)
108     for (int i = 0; i < count; i++) {
109         result_data[i] = task_data[i] * 2;
110     }
111
112     // Send the result back to the master using nonblocking send.
113     MPI_Request send_request;
114     MPI_Isend(result_data, count, MPI_INT, 0, 1, MPI_COMM_WORLD, &send_request);
115
116     // Test if the send operation has completed
117     int flag;
118     MPI_Test(&send_request, &flag, MPI_STATUS_IGNORE);
119     if (flag) {
120         printf("Worker %d finished sending result.\n", rank);
121     } else {
122         MPI_Wait(&send_request, MPI_STATUS_IGNORE);
123         printf("Worker %d finished sending result after waiting.\n", rank);
124     }
125 }
126 }
127
128 MPI_Finalize();
129 return 0;
130 }
131

```

1.5 Code Snippet #5

Run with 2 processes

```

1  #include <mpi.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <unistd.h>
5
6  int main(int argc, char **argv)
7  {
8      int rank, size;
9      MPI_Init(&argc, &argv);
10     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
11     MPI_Comm_size(MPI_COMM_WORLD, &size);
12
13     int *sendBuf = (int *)malloc(2 * sizeof(int));
14     sendBuf[0] = 5;
15     sendBuf[1] = 7;
16     int recv[2] = {0, -1};
17
18     if (rank == 0){
19         MPI_Send(sendBuf, 2, MPI_INT, 1, 0, MPI_COMM_WORLD);
20         MPI_Send(sendBuf + 1, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
21         printf("Both of the data send\n");
22     }
23     else if (rank == 1){
24         sleep(5);
25         MPI_Recv(recv, 2, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
26         MPI_Recv(recv, 2, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
27         printf("Data received in rank %d, data = %d, %d", rank, recv[0], recv[1]);
28     }
29
30     MPI_Finalize();
31     return 0;
32 }

```

1.6 Code Snippet #6

Run with 2 processes

```
1 #include <mpi.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <unistd.h>
5
6 int main(int argc, char **argv)
7 {
8     int rank, size;
9     MPI_Init(&argc, &argv);
10    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
11    MPI_Comm_size(MPI_COMM_WORLD, &size);
12
13    int *sendBuf = (int *)malloc(2 * sizeof(int));
14    sendBuf[0] = 5;
15    sendBuf[1] = 7;
16    int recv[2] = {0, -1};
17
18    if (rank == 0){
19        MPI_Send(sendBuf, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
20        MPI_Send(sendBuf + 1, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
21        printf("Both of the data send\n");
22    }
23    else if (rank == 1){
24        sleep(5);
25        MPI_Recv(recv, 2, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
26        MPI_Recv(recv, 2, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
27        printf("Data received in rank %d, data = %d, %d", rank, recv[0], recv[1]);
28    }
29
30    MPI_Finalize();
31    return 0;
32 }
```

1.7 Code Snippet #7

Run with 2 processes

```
1 #include <mpi.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <unistd.h>
5
6 int main(int argc, char **argv)
7 {
8     int rank, size;
9     MPI_Init(&argc, &argv);
10    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
11    MPI_Comm_size(MPI_COMM_WORLD, &size);
12
13    int *sendBuf = (int *)malloc(2 * sizeof(int));
14    sendBuf[0] = 5;
15    sendBuf[1] = 7;
16    int recv[2] = {0, -1};
17
18    if (rank == 0){
19        MPI_Send(sendBuf, 2, MPI_INT, 1, 0, MPI_COMM_WORLD);
20        MPI_Send(sendBuf + 1, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
21        printf("Both of the data send\n");
22    }
23    else if (rank == 1){
24        sleep(5);
25        MPI_Recv(recv, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
26        MPI_Recv(recv, 2, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
27        printf("Data received in rank %d, data = %d, %d", rank, recv[0], recv[1]);
28    }
29 }
```

```

30 MPI_Finalize();
31 return 0;
32 }

```

1.8 Code Snippet #8

Run with 3 processes

```

1
2 #include <mpi.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5
6 #define ARRAY_SIZE 16
7
8 int main(int argc, char *argv[]) {
9     MPI_Init(&argc, &argv);
10
11     int rank, size;
12     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
13     MPI_Comm_size(MPI_COMM_WORLD, &size);
14
15     int chunk_size = ARRAY_SIZE / size;
16     int *data1 = (int *)malloc(ARRAY_SIZE * sizeof(int));
17     int *data2 = (int *)malloc(ARRAY_SIZE * sizeof(int));
18     int *sub_data1 = (int *)malloc(chunk_size * sizeof(int));
19     int *sub_data2 = (int *)malloc(chunk_size * sizeof(int));
20
21     // Root initializes two arrays
22     if (rank == 0) {
23         data1 = (int *)malloc(ARRAY_SIZE * sizeof(int));
24         data2 = (int *)malloc(ARRAY_SIZE * sizeof(int));
25         for (int i = 0; i < ARRAY_SIZE; i++) {
26             data1[i] = i + 1; // 1, 2, 3, ..., 16
27             data2[i] = (i + 1) * 3; // 3, 6, 9, ..., 48
28         }
29         printf("Root initialized data1: ");
30         for (int i = 0; i < ARRAY_SIZE; i++) printf("%d ", data1[i]);
31         printf("\n");
32
33         printf("Root initialized data2: ");
34         for (int i = 0; i < ARRAY_SIZE; i++) printf("%d ", data2[i]);
35         printf("\n");
36     }
37
38     MPI_Scatter(data1, chunk_size, MPI_INT, sub_data1, chunk_size, MPI_INT, 0,
39               MPI_COMM_WORLD);
40
41     int threshold = 10;
42     MPI_Bcast(&threshold, 1, MPI_INT, 0, MPI_COMM_WORLD);
43
44     for (int i = 0; i < chunk_size; i++) {
45         if (sub_data1[i] > threshold) sub_data1[i] *= 2;
46     }
47
48     MPI_Gather(sub_data1, chunk_size, MPI_INT, data1, chunk_size, MPI_INT, 0, MPI_COMM_WORLD);
49
50     if (rank == 0) {
51         printf("Root gathered modified data1: ");
52         for (int i = 0; i < ARRAY_SIZE; i++) printf("%d ", data1[i]);
53         printf("\n");
54     }
55
56     MPI_Barrier(MPI_COMM_WORLD);
57     MPI_Bcast(data1, ARRAY_SIZE, MPI_INT, 0, MPI_COMM_WORLD);
58
59     MPI_Scatter(data2, chunk_size, MPI_INT, sub_data2, chunk_size, MPI_INT, 0,
60               MPI_COMM_WORLD);

```



```

60     int local_sum1 = 0, local_sum2 = 0;
61     for (int i = 0; i < chunk_size; i++) {
62         local_sum1 += sub_data1[i];
63         local_sum2 += sub_data2[i];
64     }
65
66     int total_sum1 = 0, total_sum2 = 0;
67     MPI_Reduce(&local_sum1, &total_sum1, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
68     MPI_Reduce(&local_sum2, &total_sum2, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
69
70     MPI_Bcast(&total_sum1, 1, MPI_INT, 0, MPI_COMM_WORLD);
71     MPI_Bcast(&total_sum2, 1, MPI_INT, 0, MPI_COMM_WORLD);
72
73     int recv_val;
74     MPI_Request send_request, recv_request;
75     int next = (rank + 1) % size;
76     int prev = (rank - 1 + size) % size;
77
78     MPI_Isend(&local_sum1, 1, MPI_INT, next, 0, MPI_COMM_WORLD, &send_request);
79     MPI_Irecv(&recv_val, 1, MPI_INT, prev, 0, MPI_COMM_WORLD, &recv_request);
80
81     MPI_Wait(&send_request, MPI_STATUS_IGNORE);
82     MPI_Wait(&recv_request, MPI_STATUS_IGNORE);
83
84     printf("Process %d received value %d from process %d\n", rank, recv_val, prev);
85
86     MPI_Finalize();
87     return 0;
88 }

```

1.9 Code Snippet #9

Run with 6 processes

```

1
2 #include <mpi.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5
6 #define DATA_SIZE 12 // Total size of the dataset
7 #define CONFIG_VALUE 5 // Configuration parameter to broadcast
8
9 int main(int argc, char **argv) {
10     MPI_Init(&argc, &argv);
11
12     int rank, size;
13     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
14     MPI_Comm_size(MPI_COMM_WORLD, &size);
15
16     // Check if the number of processes divides the data size evenly
17     if (DATA_SIZE % size != 0) {
18         if (rank == 0) {
19             fprintf(stderr, "DATA_SIZE must be divisible by the number of processes.\n");
20         }
21         MPI_Abort(MPI_COMM_WORLD, 1);
22     }
23
24     int local_size = DATA_SIZE / size;
25     int *local_data = (int *)malloc(local_size * sizeof(int));
26     int *global_data = NULL;
27     int config_value;
28
29     // Root process initializes the global dataset
30     if (rank == 0) {
31         global_data = (int *)malloc(DATA_SIZE * sizeof(int));
32         for (int i = 0; i < DATA_SIZE; i++) {
33             global_data[i] = i + 1; // Initialize with values 1, 2, 3, ..., DATA_SIZE
34         }
35     }
36

```

```

37 MPI_Bcast(&config_value, 1, MPI_INT, 0, MPI_COMM_WORLD);
38 if (rank == 0) {
39     config_value = CONFIG_VALUE;
40 }
41 MPI_Bcast(&config_value, 1, MPI_INT, 0, MPI_COMM_WORLD);
42 printf("Process %d received config_value = %d\n", rank, config_value);
43
44 MPI_Scatter(global_data, local_size, MPI_INT, local_data, local_size, MPI_INT, 0,
45     MPI_COMM_WORLD);
46
47 for (int i = 0; i < local_size; i++) {
48     local_data[i] *= config_value;
49 }
50
51 MPI_Gather(local_data, local_size, MPI_INT, global_data, local_size, MPI_INT, 0,
52     MPI_COMM_WORLD);
53
54 if (rank == 0) {
55     printf("Final result gathered by root process:\n");
56     for (int i = 0; i < DATA_SIZE; i++) {
57         printf("%d ", global_data[i]);
58     }
59     printf("\n");
60 }
61
62 MPI_Finalize();
63 return 0;
64 }

```

1.10 Code Snippet #10

Run with 3 processes

```

1
2 #include <mpi.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5
6 #define ROOT 0
7
8 int main(int argc, char *argv[]) {
9     MPI_Init(&argc, &argv);
10
11     int rank, size;
12     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
13     MPI_Comm_size(MPI_COMM_WORLD, &size);
14
15     int *data = NULL, *recv_chunk = NULL, *send_counts = NULL, *displs = NULL;
16     int *gather_displs = NULL, *gather_counts = NULL;
17     int *alltoall_send = NULL, *alltoall_recv = NULL;
18
19     int total_size = size * (size + 1) / 2; // Uneven distribution formula
20     int recv_size = rank + 1; // Each process gets (rank + 1) elements
21
22     if (rank == ROOT) {
23         data = (int *)malloc(total_size * sizeof(int));
24         send_counts = (int *)malloc(size * sizeof(int));
25         displs = (int *)malloc(size * sizeof(int));
26
27         int index = 0;
28         for (int i = 0; i < size; i++) {
29             send_counts[i] = i + 1; // Process i gets i+1 elements
30             displs[i] = index;
31             index += send_counts[i];
32         }
33
34         for (int i = 0; i < total_size; i++) {
35             data[i] = i + 1; // Initialize with sequential values
36         }

```

```

37
38     printf("Root initialized data: ");
39     for (int i = 0; i < total_size; i++) {
40         printf("%d ", data[i]);
41     }
42     printf("\n");
43 }
44
45 recv_chunk = (int *)malloc(recv_size * sizeof(int));
46
47 MPI_Scatterv(data, send_counts, displs, MPI_INT, recv_chunk, recv_size, MPI_INT, ROOT,
48             MPI_COMM_WORLD);
49
50 // Barrier: Ensures all processes have received their chunks before proceeding
51 MPI_Barrier(MPI_COMM_WORLD);
52
53 for (int i = 0; i < recv_size; i++) {
54     recv_chunk[i] *= (rank + 2);
55 }
56
57 alltoall_send = (int *)malloc(size * sizeof(int));
58 alltoall_recv = (int *)malloc(size * sizeof(int));
59
60 for (int i = 0; i < size; i++) {
61     alltoall_send[i] = rank * 100 + i; // Unique values per process
62 }
63
64 MPI_Alltoall(alltoall_send, 1, MPI_INT, alltoall_recv, 1, MPI_INT, MPI_COMM_WORLD);
65
66 MPI_Barrier(MPI_COMM_WORLD);
67
68 if (rank == ROOT) {
69     gather_counts = (int *)malloc(size * sizeof(int));
70     gather_displs = (int *)malloc(size * sizeof(int));
71
72     int index = 0;
73     for (int i = 0; i < size; i++) {
74         gather_counts[i] = i + 1;
75         gather_displs[i] = index;
76         index += gather_counts[i];
77     }
78
79 MPI_Gatherv(recv_chunk, recv_size, MPI_INT, data, gather_counts, gather_displs, MPI_INT,
80             ROOT, MPI_COMM_WORLD);
81
82 if (rank == ROOT) {
83     printf("Root gathered modified data: ");
84     for (int i = 0; i < total_size; i++) {
85         printf("%d ", data[i]);
86     }
87     printf("\n");
88 }
89
90 MPI_Finalize();
91 return 0;
92 }

```

1.11 Code Snippet #11

Run with 3 processes

```
1
2 #include <mpi.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5
6 #define DATA_SIZE 12 // Total size of the dataset
7
8 int main(int argc, char **argv) {
9     MPI_Init(&argc, &argv);
10
11     int rank, size;
12     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
13     MPI_Comm_size(MPI_COMM_WORLD, &size);
14
15     if (DATA_SIZE % size != 0) {
16         if (rank == 0) {
17             fprintf(stderr, "DATA_SIZE must be divisible by the number of processes.\n");
18         }
19         MPI_Abort(MPI_COMM_WORLD, 1);
20     }
21
22     int local_size = DATA_SIZE / size;
23     int *local_data = (int *)malloc(local_size * sizeof(int));
24     int *global_data = NULL;
25     int *send_counts = NULL;
26     int *displs = NULL;
27
28     if (rank == 0) {
29         global_data = (int *)malloc(DATA_SIZE * sizeof(int));
30         send_counts = (int *)malloc(size * sizeof(int));
31         displs = (int *)malloc(size * sizeof(int));
32
33         // Initialize global dataset with values 1, 2, 3, ..., DATA_SIZE
34         for (int i = 0; i < DATA_SIZE; i++) {
35             global_data[i] = i + 1;
36         }
37
38         for (int i = 0; i < size; i++) {
39             send_counts[i] = local_size;
40             displs[i] = i * local_size;
41         }
42     }
43
44     MPI_Scatterv(global_data, send_counts, displs, MPI_INT, local_data, local_size, MPI_INT,
45                 0, MPI_COMM_WORLD);
46
47     for (int i = 0; i < local_size; i++) {
48         local_data[i] *= (rank + 1);
49     }
50
51     MPI_Barrier(MPI_COMM_WORLD);
52
53     MPI_Gatherv(local_data, local_size, MPI_INT, global_data, send_counts, displs, MPI_INT,
54                 0, MPI_COMM_WORLD);
55
56     if (rank == 0) {
57         printf("Final result gathered by root process:\n");
58         for (int i = 0; i < DATA_SIZE; i++) {
59             printf("%d ", global_data[i]);
60         }
61         printf("\n");
62
63         int *send_buffer = (int *)malloc(size * sizeof(int));
64         int *recv_buffer = (int *)malloc(size * sizeof(int));
65
66         for (int i = 0; i < size; i++) {
```

```

66     send_buffer[i] = rank + 1;
67 }
68
69 MPI_Alltoall(send_buffer, 1, MPI_INT, recv_buffer, 1, MPI_INT, MPI_COMM_WORLD);
70
71 printf("Process %d received: ", rank);
72 for (int i = 0; i < size; i++) {
73     printf("%d ", recv_buffer[i]);
74 }
75 printf("\n");
76
77 MPI_Finalize();
78 return 0;
79 }

```

1.12 Code Snippet #12

Run with 3 processes

```

1
2 #include <mpi.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5
6 #define ROOT 0
7
8 int main(int argc, char *argv[]) {
9     MPI_Init(&argc, &argv);
10
11     int rank, size;
12     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
13     MPI_Comm_size(MPI_COMM_WORLD, &size);
14
15     int *send_data = NULL, *recv_chunk = NULL;
16     int *send_counts = NULL, *send_displs = NULL;
17     int *recv_counts = NULL, *recv_displs = NULL;
18     int *gather_counts = NULL, *gather_displs = NULL;
19
20     int total_size = size * (size + 1) / 2; // Uneven data distribution
21     int recv_size = (rank + 1) * 2; // Randomized recv size
22
23     if (rank == ROOT) {
24         send_data = (int *)malloc(total_size * sizeof(int));
25         send_counts = (int *)malloc(size * sizeof(int));
26         send_displs = (int *)malloc(size * sizeof(int));
27
28         int index = 0;
29         for (int i = 0; i < size; i++) {
30             send_counts[i] = (i + 1) * 2;
31             send_displs[i] = index;
32             index += send_counts[i];
33         }
34
35         for (int i = 0; i < total_size; i++) {
36             send_data[i] = i + 1;
37         }
38
39         printf("Root initialized data: ");
40         for (int i = 0; i < total_size; i++) {
41             printf("%d ", send_data[i]);
42         }
43         printf("\n");
44     }
45
46     recv_chunk = (int *)malloc(recv_size * sizeof(int));
47
48     MPI_Scatterv(send_data, send_counts, send_displs, MPI_INT, recv_chunk, recv_size,
49                 MPI_INT, ROOT, MPI_COMM_WORLD);
50
51     MPI_Barrier(MPI_COMM_WORLD); // Sync before modifications

```

```

51
52     for (int i = 0; i < recv_size; i++) {
53         recv_chunk[i] *= (rank + 1) * 3;
54     }
55
56     int *alltoall_send = (int *)malloc(size * size * sizeof(int));
57     int *alltoall_recv = (int *)malloc(size * size * sizeof(int));
58
59     send_counts = (int *)malloc(size * sizeof(int));
60     send_displs = (int *)malloc(size * sizeof(int));
61     recv_counts = (int *)malloc(size * sizeof(int));
62     recv_displs = (int *)malloc(size * sizeof(int));
63
64     int send_index = 0;
65     for (int i = 0; i < size; i++) {
66         send_counts[i] = (rank + 1);
67         send_displs[i] = send_index;
68         send_index += send_counts[i];
69
70         recv_counts[i] = (i + 1);
71         recv_displs[i] = (i == 0) ? 0 : recv_displs[i - 1] + recv_counts[i - 1];
72     }
73
74     for (int i = 0; i < send_index; i++) {
75         alltoall_send[i] = rank * 100 + i;
76     }
77
78     MPI_Alltoallv(alltoall_send, send_counts, send_displs, MPI_INT, alltoall_recv,
79                 recv_counts, recv_displs, MPI_INT, MPI_COMM_WORLD);
80
81     MPI_Barrier(MPI_COMM_WORLD); // Sync before gathering
82
83     if (rank == ROOT) {
84         gather_counts = (int *)malloc(size * sizeof(int));
85         gather_displs = (int *)malloc(size * sizeof(int));
86
87         int index = 0;
88         for (int i = 0; i < size; i++) {
89             gather_counts[i] = (i + 1) * 2;
90             gather_displs[i] = index;
91             index += gather_counts[i];
92         }
93
94         // Gather all modified data
95         MPI_Gatherv(recv_chunk, recv_size, MPI_INT, send_data, gather_counts, gather_displs,
96                   MPI_INT, ROOT, MPI_COMM_WORLD);
97
98         if (rank == ROOT) {
99             printf("Root gathered modified data: ");
100             for (int i = 0; i < total_size; i++) {
101                 printf("%d ", send_data[i]);
102             }
103             printf("\n");
104         }
105
106         MPI_Finalize();
107         return 0;
108     }

```

1.13 Code Snippet #13

Run with 4 processes

```
1
2 #include <mpi.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5
6 #define N 4 // Matrix size
7 int main(int argc, char **argv) {
8     MPI_Init(&argc, &argv);
9
10    int rank, size;
11    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
12    MPI_Comm_size(MPI_COMM_WORLD, &size);
13
14    // Each process handles part of the matrix
15    int rows_per_proc = N / size;
16    int local_matrix[rows_per_proc][N];
17    int vector[N];
18    int local_result[rows_per_proc];
19    int final_result[N];
20
21    if (rank == 0) {
22        int full_matrix[N][N] = {
23            {1, 2, 3, 4},
24            {5, 6, 7, 8},
25            {9, 10, 11, 12},
26            {13, 14, 15, 16}
27        };
28        int full_vector[N] = {1, 1, 1, 1};
29
30        MPI_Scatter(full_matrix, rows_per_proc * N, MPI_INT, local_matrix, rows_per_proc * N,
31                    MPI_INT, 0, MPI_COMM_WORLD);
32
33        MPI_Bcast(full_vector, N, MPI_INT, 0, MPI_COMM_WORLD);
34
35        for (int i = 0; i < N; i++) {
36            vector[i] = full_vector[i];
37        }
38    } else {
39        MPI_Scatter(NULL, rows_per_proc * N, MPI_INT, local_matrix, rows_per_proc * N,
40                    MPI_INT, 0, MPI_COMM_WORLD);
41        MPI_Bcast(vector, N, MPI_INT, 0, MPI_COMM_WORLD);
42    }
43
44    for (int i = 0; i < rows_per_proc; i++) {
45        local_result[i] = 0;
46        for (int j = 0; j < N; j++) {
47            local_result[i] += local_matrix[i][j] * vector[j];
48        }
49    }
50
51    MPI_Gather(local_result, rows_per_proc, MPI_INT, final_result, rows_per_proc, MPI_INT,
52              0, MPI_COMM_WORLD);
53
54    int total_sum;
55    MPI_Allreduce(&local_result, &total_sum, rows_per_proc, MPI_INT, MPI_SUM, MPI_COMM_WORLD);
56
57    if (rank == 0) {
58        for (int i = 0; i < N; i++)
59            printf("%d ", final_result[i]);
60        printf("\n");
61        printf("Root Got: %d\n", total_sum);
62    }
63    MPI_Finalize();
64    return 0;
65 }
```

1.14 Code Snippet #14

Run with 4 processes

```
1
2 #include <mpi.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5
6 int main(int argc, char **argv) {
7     MPI_Init(&argc, &argv);
8
9     int rank, size;
10    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
11    MPI_Comm_size(MPI_COMM_WORLD, &size);
12
13    // Ensure we have at least 3 processes
14    if (size < 3) {
15        fprintf(stderr, "Run with at least 3 processes!\n");
16        MPI_Abort(MPI_COMM_WORLD, 1);
17    }
18
19    int send_value = rank + 1; // Each process sends its rank + 1
20    int *allgather_result = (int *)malloc(size * sizeof(int));
21
22    MPI_Allgather(&send_value, 1, MPI_INT, allgather_result, 1, MPI_INT, MPI_COMM_WORLD);
23
24    printf("Rank %d - Allgather result: ", rank);
25    for (int i = 0; i < size; i++) {
26        printf("%d ", allgather_result[i]);
27    }
28    printf("\n");
29
30    int local_size = rank + 1;
31    int *send_buffer = (int *)malloc(local_size * sizeof(int));
32    for (int i = 0; i < local_size; i++)
33        send_buffer[i] = rank * 10 + i;
34
35    int *recv_counts = (int *)malloc(size * sizeof(int));
36    MPI_Allgather(&local_size, 1, MPI_INT, recv_counts, 1, MPI_INT, MPI_COMM_WORLD);
37
38    int *displacements = (int *)malloc(size * sizeof(int));
39    displacements[0] = 0;
40    int total_size = recv_counts[0];
41    for (int i = 1; i < size; i++) {
42        displacements[i] = displacements[i - 1] + recv_counts[i - 1];
43        total_size += recv_counts[i];
44    }
45
46    int *allgatherv_result = (int *)malloc(total_size * sizeof(int));
47
48    MPI_Allgatherv(send_buffer, local_size, MPI_INT, allgatherv_result, recv_counts,
49        displacements, MPI_INT, MPI_COMM_WORLD);
50
51    printf("Rank %d - Allgatherv result: ", rank);
52    for (int i = 0; i < total_size; i++) {
53        printf("%d ", allgatherv_result[i]);
54    }
55    printf("\n");
56
57    int *send_alltoall = (int *)malloc(size * sizeof(int));
58    int *recv_alltoall = (int *)malloc(size * sizeof(int));
59
60    for (int i = 0; i < size; i++) send_alltoall[i] = rank * 100 + i;
61
62    MPI_Alltoall(send_alltoall, 1, MPI_INT, recv_alltoall, 1, MPI_INT, MPI_COMM_WORLD);
63
64    printf("Rank %d - Alltoall result: ", rank);
65    for (int i = 0; i < size; i++) {
66        printf("%d ", recv_alltoall[i]);
67    }
68}
```



```

67     printf("\n");
68
69     int *send_counts = (int *)malloc(size * sizeof(int));
70     int *recv_counts_v = (int *)malloc(size * sizeof(int));
71     int *send_displs = (int *)malloc(size * sizeof(int));
72     int *recv_displs = (int *)malloc(size * sizeof(int));
73
74     for (int i = 0; i < size; i++) {
75         send_counts[i] = (rank + i) % size + 1;
76         recv_counts_v[i] = (i + rank) % size + 1;
77     }
78
79     send_displs[0] = recv_displs[0] = 0;
80     for (int i = 1; i < size; i++) {
81         send_displs[i] = send_displs[i - 1] + send_counts[i - 1];
82         recv_displs[i] = recv_displs[i - 1] + recv_counts_v[i - 1];
83     }
84
85     int send_total = send_displs[size - 1] + send_counts[size - 1];
86     int recv_total = recv_displs[size - 1] + recv_counts_v[size - 1];
87
88     int *send_data_v = (int *)malloc(send_total * sizeof(int));
89     int *recv_data_v = (int *)malloc(recv_total * sizeof(int));
90
91     for (int i = 0; i < send_total; i++) send_data_v[i] = rank * 10 + i;
92
93     MPI_Alltoallv(send_data_v, send_counts, send_displs, MPI_INT, recv_data_v, recv_counts_v,
94         , recv_displs, MPI_INT, MPI_COMM_WORLD);
95
96     printf("Rank %d - Alltoallv result: ", rank);
97     for (int i = 0; i < recv_total; i++) {
98         printf("%d ", recv_data_v[i]);
99     }
100     printf("\n");
101
102     MPI_Finalize();
103     return 0;
104 }

```

1.15 Code Snippet #15

Run with 4 processes

```

1
2 #include <mpi.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5
6 int main(int argc, char** argv) {
7     MPI_Init(&argc, &argv);
8     int rank, size;
9     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
10    MPI_Comm_size(MPI_COMM_WORLD, &size);
11
12    // Initial clues (rank 0 starts with 9)
13    int clues = (rank == 0) ? 9 : 0;
14    int fear = rank + 2; // Base fear level
15
16    // 1. MPI_Scatter: Distribute clues
17    int* clue_buf = (rank == 0) ? (int*)malloc(size * sizeof(int)) : NULL;
18    if (rank == 0) {
19        for (int i = 0; i < size; i++) clue_buf[i] = 2 + (i % 2); // 2 or 3
20    }
21    int local_clues;
22    MPI_Scatter(clue_buf, 1, MPI_INT, &local_clues, 1, MPI_INT, 0, MPI_COMM_WORLD);
23    clues = local_clues;
24    fear += clues * 2; // Fear rises with clues
25
26    // 2. MPI_Reduce: Total clues
27    int total_clues = 0;

```

```

28 MPI_Reduce(&clues, &total_clues, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
29
30 // 3. MPI_Bcast: Share total clues
31 MPI_Bcast(&total_clues, 1, MPI_INT, 0, MPI_COMM_WORLD);
32
33 // 4. MPI_Alltoall: Trade fear levels
34 int* fear_send = (int*)malloc(size * sizeof(int));
35 for (int i = 0; i < size; i++) fear_send[i] = fear / (size - i);
36 int* fear_rcv = (int*)malloc(size * sizeof(int));
37 MPI_Alltoall(fear_send, 1, MPI_INT, fear_rcv, 1, MPI_INT, MPI_COMM_WORLD);
38 for (int i = 0; i < size; i++) fear += fear_rcv[i];
39
40 // 5. MPI_Gather: Collect fear levels
41 int* all_fears = (int*)malloc(size * sizeof(int));
42 MPI_Gather(&fear, 1, MPI_INT, all_fears, 1, MPI_INT, 0, MPI_COMM_WORLD);
43
44 // 6. MPI_Allreduce: Max fear
45 int max_fear = 0;
46 MPI_Allreduce(&fear, &max_fear, 1, MPI_INT, MPI_MAX, MPI_COMM_WORLD);
47
48 // Output
49 printf("Rank %d: Clues %d, Fear %d, Max %d\n", rank, clues, fear, max_fear);
50 if (rank == 0) printf("Total Clues: %d\n", total_clues);
51
52 // Cleanup
53 free(fear_send); free(fear_rcv); free(all_fears);
54 if (rank == 0) free(clue_buf);
55
56 MPI_Finalize();
57 return 0;
58 }

```

1.16 Code Snippet #16

Run with 4 processes

```

1
2 #include <mpi.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5
6 int main(int argc, char** argv) {
7     MPI_Init(&argc, &argv);
8     int rank, size;
9     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
10    MPI_Comm_size(MPI_COMM_WORLD, &size);
11
12    // Initial rumor value (starts at rank 0)
13    int rumor = (rank == 0) ? 10 : 0; // Rumor starts as 10
14    int heard = (rank == 0) ? 1 : 0; // 1 if heard, 0 if not
15
16    // Step 1: Spread rumor locally (exaggerate if heard)
17    if (heard) {
18        rumor += rank * 2; // Exaggerate based on rank
19    }
20
21    // 2. MPI_Reduce: Count total who heard (to root)
22    int total_heard = 0;
23    MPI_Reduce(&heard, &total_heard, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
24
25    // 3. MPI_Allreduce: Max rumor value across all
26    int global_max_rumor = 0;
27    MPI_Allreduce(&rumor, &global_max_rumor, 1, MPI_INT, MPI_MAX, MPI_COMM_WORLD);
28
29    // Step 4: Spread rumor to all via MPI_Alltoall
30    int* send_rumor = (int*)malloc(size * sizeof(int));
31    for (int i = 0; i < size; i++) {
32        send_rumor[i] = rumor + (i % 2 ? 1 : -1); // Slight variation per target
33    }
34    int* rcv_rumor = (int*)malloc(size * sizeof(int));

```

```

35 MPI_Alltoall(send_rumor, 1, MPI_INT, recv_rumor, 1, MPI_INT, MPI_COMM_WORLD);
36
37 // Update rumor: Take max received value if not heard yet
38 if (!heard) {
39     rumor = recv_rumor[0];
40     for (int i = 1; i < size; i++) {
41         if (recv_rumor[i] > rumor) rumor = recv_rumor[i];
42     }
43     heard = 1;
44 }
45
46 // 5. MPI_Barrier: Sync before final output
47 MPI_Barrier(MPI_COMM_WORLD);
48
49 printf("Rank %d: Rumor %d, Heard %d, Global Max %d, Recv [",
50        rank, rumor, heard, global_max_rumor);
51 for (int i = 0; i < size; i++) printf("%d ", recv_rumor[i]);
52 printf("]\n");
53 if (rank == 0) printf("Total Heard: %d\n", total_heard);
54
55 // Cleanup
56 free(send_rumor); free(recv_rumor);
57
58 MPI_Finalize();
59 return 0;
60 }

```

1.17 Code Snippet #17

Run with 3 processes

```

1
2 #include <mpi.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5
6 int main(int argc, char** argv) {
7     MPI_Init(&argc, &argv);
8     int rank, size;
9     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
10    MPI_Comm_size(MPI_COMM_WORLD, &size);
11
12    // Initial state: UFOs land at rank 0
13    int ufos = (rank == 0) ? 3 : 0; // 3 UFOs start at rank 0
14    int aliens = ufos * (rank + 1); // Aliens spawned: UFOs * (rank+1)
15
16    // 1. MPI_Reduce: Total aliens to root
17    int total_alien = 0;
18    MPI_Reduce(&aliens, &total_alien, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
19
20    // 2. MPI_Scatter: Distribute UFOs from root
21    int* ufo_buffer = (rank == 0) ? (int*)malloc(size * sizeof(int)) : NULL;
22    if (rank == 0) {
23        for (int i = 0; i < size; i++) ufo_buffer[i] = 1 + (i % 2); // 1 or 2 UFOs
24    }
25
26    int local_ufos;
27    MPI_Scatter(ufo_buffer, 1, MPI_INT, &local_ufos, 1, MPI_INT, 0, MPI_COMM_WORLD);
28    aliens += local_ufos * 2; // Each UFO spawns 2 more aliens
29
30    // 3. MPI_Gather: Collect alien counts
31    int* all_alien = (int*)malloc(size * sizeof(int));
32    MPI_Gather(&aliens, 1, MPI_INT, all_alien, 1, MPI_INT, 0, MPI_COMM_WORLD);
33
34    // 4. MPI_Alltoall: Spread aliens to neighbors
35    int* send_alien = (int*)malloc(size * sizeof(int));
36    for (int i = 0; i < size; i++) send_alien[i] = all_alien[i] / size + (i % 2);
37    int* recv_alien = (int*)malloc(size * sizeof(int));
38    MPI_Alltoall(send_alien, 1, MPI_INT, recv_alien, 1, MPI_INT, MPI_COMM_WORLD);
39

```

```

40 // Update aliens with received spread
41 int new_aliens = aliens;
42 for (int i = 0; i < size; i++) new_aliens += recv_aliens[i];
43
44 // 5. MPI_Barrier: Sync before stats
45 MPI_Barrier(MPI_COMM_WORLD);
46
47 // 6. MPI_Allreduce: Max alien force
48 int max_aliens = 0;
49 MPI_Allreduce(&new_aliens, &max_aliens, 1, MPI_INT, MPI_MAX, MPI_COMM_WORLD);
50
51 // Output
52 printf("Rank %d: UFOs %d, Aliens %d, Max Force %d\n",
53        rank, local_ufos, new_aliens, max_aliens);
54 if (rank == 0) {
55     printf("Total Aliens: %d, All Counts [", total_aliens);
56     for (int i = 0; i < size; i++) printf("%d ", all_aliens[i]);
57     printf("]\n");
58 }
59
60 // Cleanup
61 free(all_aliens); free(send_aliens); free(recv_aliens);
62 if (rank == 0) free(ufo_buffer);
63
64 MPI_Finalize();
65 return 0;
66 }

```

1.18 Code Snippet #18

Run with 4 processes

```

1
2 #include <mpi.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5
6 int main(int argc, char** argv) {
7     MPI_Init(&argc, &argv);
8     int rank, size;
9     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
10    MPI_Comm_size(MPI_COMM_WORLD, &size);
11
12    // Initial swords (rank 0 starts with 10)
13    int swords = (rank == 0) ? 10 : 0;
14    int strength = rank + 3; // Base strength
15
16    // 1. MPI_Scatter: Distribute swords
17    int* sword_buf = (rank == 0) ? (int*)malloc(size * sizeof(int)) : NULL;
18    if (rank == 0) {
19        for (int i = 0; i < size; i++) sword_buf[i] = 3 - (i % 2); // 3 or 2
20    }
21    int local_swords;
22    MPI_Scatter(sword_buf, 1, MPI_INT, &local_swords, 1, MPI_INT, 0, MPI_COMM_WORLD);
23    swords = local_swords;
24    strength += swords; // Strength from swords
25
26    // 2. MPI_Reduce: Total dragons slain (swords = dragons)
27    int total_dragons = 0;
28    MPI_Reduce(&swords, &total_dragons, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
29
30    // 3. MPI_Bcast: Share total dragons
31    MPI_Bcast(&total_dragons, 1, MPI_INT, 0, MPI_COMM_WORLD);
32    strength += total_dragons / size; // Boost from fame
33
34    // 4. MPI_Alltoall: Trade dragon scales
35    int* scale_send = (int*)malloc(size * sizeof(int));
36    for (int i = 0; i < size; i++) scale_send[i] = strength / (i + 2);
37    int* scale_recv = (int*)malloc(size * sizeof(int));
38    MPI_Alltoall(scale_send, 1, MPI_INT, scale_recv, 1, MPI_INT, MPI_COMM_WORLD);

```

```

39     for (int i = 0; i < size; i++) strength += scale_recv[i];
40
41     // 5. MPI_Gather: Collect strengths
42     int* all_strengths = (int*)malloc(size * sizeof(int));
43     MPI_Gather(&strength, 1, MPI_INT, all_strengths, 1, MPI_INT, 0, MPI_COMM_WORLD);
44
45     // 6. MPI_Allreduce: Max strength
46     int max_strength = 0;
47     MPI_Allreduce(&strength, &max_strength, 1, MPI_INT, MPI_MAX, MPI_COMM_WORLD);
48
49     // Output
50     printf("Rank %d: Swords %d, Strength %d, Max %d\n", rank, swords, strength, max_strength);
51     if (rank == 0) printf("Total Dragons: %d\n", total_dragons);
52
53     // Cleanup
54     free(scale_send); free(scale_recv); free(all_strengths);
55     if (rank == 0) free(sword_buf);
56
57     MPI_Finalize();
58     return 0;
59 }

```

1.19 Code Snippet #19

Run with 3 processes

```

1
2 #include <mpi.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5
6 int main(int argc, char** argv) {
7     MPI_Init(&argc, &argv);
8     int rank, size;
9     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
10    MPI_Comm_size(MPI_COMM_WORLD, &size);
11
12    // Initial fuel (rank 0 starts with 12)
13    int fuel = (rank == 0) ? 12 : 0;
14    int speed = rank + 1; // Base speed per ship
15
16    // 1. MPI_Scatter: Distribute fuel
17    int* fuel_buf = (rank == 0) ? (int*)malloc(size * sizeof(int)) : NULL;
18    if (rank == 0) {
19        for (int i = 0; i < size; i++) fuel_buf[i] = 3 + (i % 2); // 3 or 4
20    }
21    int local_fuel;
22    MPI_Scatter(fuel_buf, 1, MPI_INT, &local_fuel, 1, MPI_INT, 0, MPI_COMM_WORLD);
23    fuel = local_fuel;
24    speed += fuel; // Speed increases with fuel
25
26    // 2. MPI_Bcast: Share total fuel
27    int total_fuel = fuel;
28    MPI_Reduce(&fuel, &total_fuel, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
29    MPI_Bcast(&total_fuel, 1, MPI_INT, 0, MPI_COMM_WORLD);
30
31    // 3. MPI_Alltoall: Trade speed boosts
32    int* trade_send = (int*)malloc(size * sizeof(int));
33    for (int i = 0; i < size; i++) trade_send[i] = speed / (i + 1);
34    int* trade_recv = (int*)malloc(size * sizeof(int));
35    MPI_Alltoall(trade_send, 1, MPI_INT, trade_recv, 1, MPI_INT, MPI_COMM_WORLD);
36    for (int i = 0; i < size; i++) speed += trade_recv[i];
37
38    // 4. MPI_Gather: Collect speeds
39    int* all_speeds = (int*)malloc(size * sizeof(int));
40    MPI_Gather(&speed, 1, MPI_INT, all_speeds, 1, MPI_INT, 0, MPI_COMM_WORLD);
41
42    // 5. MPI_Allreduce: Max speed
43    int max_speed = 0;

```

```

44 MPI_Allreduce(&speed, &max_speed, 1, MPI_INT, MPI_MAX, MPI_COMM_WORLD);
45
46 // Output
47 printf("Rank %d: Fuel %d, Speed %d, Max %d\n", rank, fuel, speed, max_speed);
48 if (rank == 0) printf("Total Fuel: %d\n", total_fuel);
49
50 // Cleanup
51 free(trade_send); free(trade_recv); free(all_speeds);
52 if (rank == 0) free(fuel_buf);
53
54 MPI_Finalize();
55 return 0;
56 }

```

1.20 Code Snippet #20

Run with 4 processes

```

1
2 #include <mpi.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5
6 int main(int argc, char** argv) {
7     MPI_Init(&argc, &argv);
8     int rank, size;
9     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
10    MPI_Comm_size(MPI_COMM_WORLD, &size);
11
12    // Initial treasures (rank 0 starts with 5)
13    int treasures = (rank == 0) ? 5 : 0;
14
15    // Rank 0 sends treasures to rank 1
16    if (rank == 0) {
17        MPI_Send(&treasures, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
18    }
19    if (rank == 1) {
20        MPI_Recv(&treasures, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
21        treasures += 1; // Rank 1 adds a find
22    }
23
24    // Allocate memory for sendcounts and displs
25    int* sendcounts = (int*)malloc(size * sizeof(int));
26    int* displs = (int*)malloc(size * sizeof(int));
27    int total = 0;
28
29    // Initialize sendcounts and displs on rank 0
30    if (rank == 0) {
31        for (int i = 0; i < size; i++) {
32            sendcounts[i] = 1 + (i % 2); // 1 or 2 treasures
33            displs[i] = total;
34            total += sendcounts[i];
35        }
36    }
37
38    // Broadcast sendcounts and displs to all ranks
39    MPI_Bcast(sendcounts, size, MPI_INT, 0, MPI_COMM_WORLD);
40    MPI_Bcast(displs, size, MPI_INT, 0, MPI_COMM_WORLD);
41
42    // Allocate memory for local_treasures
43    int* local_treasures = (int*)malloc(sendcounts[rank] * sizeof(int));
44
45    if (rank == 0) {
46        int* treasures_array = (int*)malloc(total * sizeof(int));
47        for (int i = 0; i < total; i++) {
48            treasures_array[i] = treasures; // Fill with initial treasures
49        }
50        MPI_Scatterv(treasures_array, sendcounts, displs, MPI_INT, local_treasures,
51                    sendcounts[rank], MPI_INT, 0, MPI_COMM_WORLD);
52        free(treasures_array);

```

```

52 } else {
53     MPI_Scatterv(NULL, sendcounts, displs, MPI_INT, local_treasures, sendcounts[rank],
54                 MPI_INT, 0, MPI_COMM_WORLD);
55 }
56 for (int i = 0; i < sendcounts[rank]; i++) {
57     treasures += local_treasures[i];
58 }
59
60 // Reduce total treasures
61 int total_treasures = 0;
62 MPI_Reduce(&treasures, &total_treasures, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
63
64 int* trade_send = (int*)malloc(size * sizeof(int));
65 int* trade_recv = (int*)malloc(size * sizeof(int));
66 for (int i = 0; i < size; i++) {
67     trade_send[i] = treasures / (size - i);
68 }
69 MPI_Alltoall(trade_send, 1, MPI_INT, trade_recv, 1, MPI_INT, MPI_COMM_WORLD);
70
71 for (int i = 0; i < size; i++) {
72     treasures += trade_recv[i];
73 }
74
75 int* all_treasures = (int*)malloc(total * sizeof(int));
76 MPI_Gatherv(&treasures, sendcounts[rank], MPI_INT, all_treasures, sendcounts, displs,
77             MPI_INT, 0, MPI_COMM_WORLD);
78
79 int max_treasures = 0;
80 MPI_Allreduce(&treasures, &max_treasures, 1, MPI_INT, MPI_MAX, MPI_COMM_WORLD);
81
82 printf("Rank %d: Treasures %d, Max %d\n", rank, treasures, max_treasures);
83 if (rank == 0) {
84     printf("Total Treasures: %d, [", total_treasures);
85     for (int i = 0; i < total; i++) {
86         printf("%d ", all_treasures[i]);
87     }
88     printf("]\n");
89 }
90 MPI_Finalize();
91 return 0;
92 }

```

2 Expected Outputs

Below are the expected outputs for each code snippet, presented in a formatted manner for clarity.

Output for Code Snippet #1

```
Both of the data send
Data received in rank 1, data = 7, -1
```

Output for Code Snippet #2

```
Both of the data send
Count of data received = 0
Data received in rank 1, data = -1, -1
```

Output for Code Snippet #3

```
Process 1 :  
1 3 5  
Process 2 :  
2 5 8  
Process 0 :  
0 1 2  
Process 0 received: 0 1 2  
Process 1 received: 1 3 5  
Process 2 received: 2 5 8
```

Output for Code Snippet #4

```
Master sent task 0 to worker 1  
Master sent task 1 to worker 2  
Master sent task 2 to worker 1  
Master sent task 3 to worker 2  
Master sent task 4 to worker 1  
Worker 2 received task: 4 5 6  
Master received result from worker 2: 8 10 12  
Worker 2 finished sending result.  
Worker 2 received task: 10 11 12  
Master received result from worker 2: 20 22 24  
Worker 2 finished sending result.  
Worker 1 received task: 1 2 3  
Worker 1 finished sending result.  
Worker 1 received task: 7 8 9  
Worker 1 finished sending result.  
Worker 1 received task: 13 14 15  
Worker 1 finished sending result.  
Master received result from worker 1: 2 4 6  
Master received result from worker 1: 14 16 18  
Master received result from worker 1: 26 28 30  
Master sent termination message to worker 1  
Master sent termination message to worker 2  
Worker 2 received termination signal. Exiting.  
Worker 1 received termination signal. Exiting.
```

Output for Code Snippet #5

```
Both of the data send  
Data received in rank 1, data = 7, 7
```

Output for Code Snippet #6

```
Both of the data send  
Data received in rank 1, data = 7, -1
```

Output for Code Snippet #7

```
Both of the data send  
// Error  
MPI_Recv(buf=0x7ffccaf73790, count=1, MPI_INT, src=0, tag=0, MPI_COMM_WORLD, status=0x1)  
failed MPIDI_CH3_PktHandler_EagerShortSend(363): Message from rank 0 and tag 0 truncated;  
8 bytes received but buffer size is 4
```


Output for Code Snippet #8

```
Root initialized data1: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
Root initialized data2: 3 6 9 12 15 18 21 24 27 30 33 36 39 42 45 48
Root gathered modified data1: 1 2 3 4 5 6 7 8 9 10 22 24 26 28 30 16
Process 0 received value 130 from process 2
Process 1 received value 15 from process 0
Process 2 received value 40 from process 1
```

Output for Code Snippet #9

```
Process 0 received config_value = 5
Process 1 received config_value = 5
Process 4 received config_value = 5
Process 5 received config_value = 5
Process 2 received config_value = 5
Process 3 received config_value = 5
Final result gathered by root process:
5 10 15 20 25 30 35 40 45 50 55 60
```

Output for Code Snippet #10

```
Root initialized data: 1 2 3 4 5 6
Root gathered modified data: 2 6 9 16 20 24
```

Output for Code Snippet #11

```
Final result gathered by root process:
1 2 3 4 10 12 14 16 27 30 33 36
Process 0 received: 1 2 3
Process 2 received: 1 2 3
Process 1 received: 1 2 3
```

Output for Code Snippet #12

```
Root initialized data: 1 2 3 4 5 6
Root gathered modified data: 3 6 18 24 30 36
```

Output for Code Snippet #13

```
10 26 42 58
Root Got: 136
```

Output for Code Snippet #14

```
Rank 0 - Allgather result: 1 2 3 4
Rank 1 - Allgather result: 1 2 3 4
Rank 2 - Allgather result: 1 2 3 4
Rank 3 - Allgather result: 1 2 3 4
Rank 0 - Allgatherv result: 0 10 11 20 21 22 30 31 32 33
Rank 3 - Allgatherv result: 0 10 11 20 21 22 30 31 32 33
Rank 2 - Allgatherv result: 0 10 11 20 21 22 30 31 32 33
Rank 1 - Allgatherv result: 0 10 11 20 21 22 30 31 32 33
Rank 1 - Alltoall result: 1 101 201 301
Rank 2 - Alltoall result: 2 102 202 302
Rank 0 - Alltoall result: 0 100 200 300
Rank 3 - Alltoall result: 3 103 203 303
Rank 3 - Alltoallv result: 6 7 8 9 19 28 29 37 38 39
Rank 0 - Alltoallv result: 0 10 11 20 21 22 30 31 32 33
Rank 2 - Alltoallv result: 3 4 5 15 16 17 18 27 35 36
Rank 1 - Alltoallv result: 1 2 12 13 14 23 24 25 26 34
```

Output for Code Snippet #15

```
Rank 0: Clues 2, Fear 13, Max 45
Rank 1: Clues 3, Fear 19, Max 45
Rank 2: Clues 2, Fear 24, Max 45
Rank 3: Clues 3, Fear 45, Max 45
Total Clues: 10
```

Output for Code Snippet #16

```
Rank 0: Rumor 10, Heard 1, Global Max 10, Recv [9 -1 -1 -1 ]
Rank 1: Rumor 11, Heard 1, Global Max 10, Recv [11 1 1 1 ]
Rank 2: Rumor 9, Heard 1, Global Max 10, Recv [9 -1 -1 -1 ]
Rank 3: Rumor 11, Heard 1, Global Max 10, Recv [11 1 1 1 ]
Total Heard: 1
```

Output for Code Snippet #17

```
Rank 0: UFOs 1, Aliens 7, Max Force 9
Rank 1: UFOs 2, Aliens 9, Max Force 9
Rank 2: UFOs 1, Aliens 4, Max Force 9
Total Aliens: 3, All Counts [5 4 2 ]
```

Output for Code Snippet #18

```
Rank 0: Swords 3, Strength 26, Max 26
Rank 2: Swords 3, Strength 18, Max 26
Rank 1: Swords 2, Strength 18, Max 26
Rank 3: Swords 2, Strength 16, Max 26

Total Dragons: 10
```

Output for Code Snippet #19

```
Rank 0: Fuel 3, Speed 20, Max 20
Rank 2: Fuel 3, Speed 11, Max 20
Rank 1: Fuel 4, Speed 14, Max 20
Total Fuel: 10
```

Output for Code Snippet #20

```
Rank 0: Treasures 19, Max 51  
Rank 1: Treasures 28, Max 51  
Rank 2: Treasures 25, Max 51  
Rank 3: Treasures 51, Max 51  
Total Treasures: 41, [19 28 0 25 51 0]
```